

## Unidad VI: Árboles Avanzados

### Clase 10 – Cuando el árbol se desbalancea: AVL

#### Objetivo

Entender por qué el balance es crítico en estructuras de búsqueda

#### Tema

- Problema del desbalance. Rotaciones simples y dobles. Garantía  $O(\log n)$ .

#### Árboles AVL: balance, rotaciones e implementación esencial

##### Introducción conceptual

Un Árbol Binario de Búsqueda permite organizar valores de manera jerárquica para acelerar operaciones como buscar, insertar y eliminar. Su ventaja depende de una regla simple: los valores menores que un nodo se ubican hacia la izquierda y los valores mayores hacia la derecha. Esa regla permite descartar ramas completas durante una búsqueda. Sin embargo, la eficiencia del árbol no depende solamente de que esa regla se cumpla, sino también de la forma que adopta la estructura.

Un árbol puede respetar perfectamente la propiedad de búsqueda y, aun así, volverse ineficiente si queda demasiado inclinado hacia un lado. En ese caso, la estructura se parece más a una lista enlazada que a un árbol balanceado. El Árbol AVL surge para resolver ese problema: mantiene la propiedad de un Árbol Binario de Búsqueda, pero agrega una condición de equilibrio que evita que la altura crezca de manera descontrolada. El material específico de AVL define esta estructura como un ABB al que se le añade una condición de equilibrio: para todo nodo, la altura de sus subárboles izquierdo y derecho puede diferir a lo sumo en una unidad.

##### ABB y costo dependiente de la altura

Un Árbol Binario de Búsqueda, o **ABB**, es un árbol binario cuyos elementos están ordenados de izquierda a derecha. Dado un nodo  $X$ , todos los valores del subárbol izquierdo son menores que  $X$ , y todos los valores del subárbol derecho son mayores que  $X$ . Además, en la especificación utilizada por la cátedra, el ABB no admite elementos repetidos.

La búsqueda en un ABB se realiza tomando decisiones sucesivas. Si el valor buscado es menor que el nodo actual, se continúa por el subárbol izquierdo. Si es mayor, se continúa por el subárbol derecho. Si es igual, el valor fue encontrado. Esta estrategia es eficiente cuando el árbol tiene una altura moderada, porque cada comparación reduce el espacio de búsqueda.

El costo de buscar, insertar o eliminar en un ABB se expresa como  $O(h)$ , donde  $h$  representa la altura del árbol. La altura es la distancia máxima entre la raíz y una hoja. Si el árbol está razonablemente balanceado, la altura se aproxima a  $\log n$ , siendo  $n$  la cantidad de nodos. En ese caso, el costo se acerca a  $O(\log n)$ . Pero si el árbol queda degenerado, la altura puede aproximarse a  $n$ , y el costo puede degradarse hacia  $O(n)$ .

Por ejemplo, si se insertan los valores 10, 20, 30, 40, 50 en ese orden, un ABB común ubicará cada nuevo valor a la derecha del anterior. El resultado será una estructura lineal. Buscar el valor 50 exigirá pasar por 10, luego 20, luego 30, luego 40 y finalmente 50. Aunque la propiedad del ABB se cumple, el costo real se parece al de una lista.

### Árbol AVL como ABB balanceado

Un **AVL** es un Árbol Binario de Búsqueda balanceado. La sigla proviene de los apellidos Adelson, Velskii y Landis. La idea principal es conservar la lógica del ABB, pero impedir que el árbol crezca de forma excesivamente despareja.

La regla de equilibrio de AVL establece que, para cada nodo, la diferencia de altura entre el subárbol izquierdo y el subárbol derecho no puede superar 1. Esto significa que un nodo puede tener ambos subárboles con la misma altura, o puede tener un lado una unidad más alto que el otro. Lo que no puede ocurrir es que un lado tenga dos o más niveles de diferencia respecto del otro.

La diferencia entre un ABB común y un AVL no está en la forma de comparar los valores. Ambos respetan la misma propiedad de búsqueda. La diferencia está en el control estructural. El ABB solo garantiza el orden; el AVL garantiza orden y equilibrio. Por eso, las aplicaciones del AVL son similares a las de un ABB, pero con una ventaja importante: las búsquedas quedan acotadas por una complejidad de orden  $O(\log n)$ . El documento de AVL señala que los ABB pueden tender en el peor caso hacia una estructura similar a una lista, mientras que AVL resuelve ese problema manteniendo las búsquedas acotadas en  $O(\log n)$ .

### Altura y factor de equilibrio

La **altura** de un nodo es la mayor distancia desde ese nodo hasta una hoja. En términos operativos, permite estimar cuántos pasos puede requerir una operación que desciende por el árbol. La raíz tiene como altura la longitud del camino más largo hasta una hoja. Un nodo hoja puede considerarse de altura 0 si se mide en cantidad de aristas, o de altura 1 si se mide en cantidad de nodos. Ambas convenciones son válidas, pero debe mantenerse una sola durante toda la implementación.

El **factor de equilibrio** es el valor que permite diagnosticar si un nodo cumple la regla AVL. Para trabajar con una convención clara, puede definirse así:

$$FE = \text{altura}(\text{subárbol derecho}) - \text{altura}(\text{subárbol izquierdo})$$

Con esta convención, un nodo está balanceado si su factor de equilibrio es -1, 0 o +1.

Un factor 0 indica que ambos subárboles tienen la misma altura. Un factor +1 indica que el subárbol derecho es una unidad más alto. Un factor -1 indica que el subárbol izquierdo es una unidad más alto. Cuando aparece +2 o -2, el nodo está desbalanceado y debe corregirse.

Conviene diferenciar con precisión tres conceptos que suelen confundirse. La altura mide profundidad estructural. El factor de equilibrio compara las alturas de dos subárboles. El balance es la condición que se cumple cuando ese factor se mantiene dentro del rango permitido. No alcanza con decir que un árbol “parece equilibrado”; en AVL el equilibrio se decide nodo por nodo, mediante una regla concreta.

### Inserción en AVL

La inserción en AVL comienza igual que la inserción en un ABB. El nuevo valor se compara con la raíz, se baja por izquierda o derecha según corresponda y se ubica en la posición que respeta la propiedad de búsqueda. La diferencia aparece después de insertar el nuevo nodo.

Una vez realizada la inserción, se deben revisar los nodos ubicados en el camino desde el nodo insertado hacia la raíz. En cada uno se actualiza la altura y se calcula el factor de equilibrio. Si todos los factores permanecen en -1, 0 o +1, no es necesario modificar la estructura. Si aparece un nodo con factor +2 o -2, el árbol necesita una rotación.

El punto clave es que la corrección no se realiza sobre todo el árbol. Se identifica el primer nodo desbalanceado al volver desde el punto de inserción hacia la raíz, y se corrige localmente ese subárbol. Esta característica permite que AVL mantenga la eficiencia sin reconstruir la estructura completa.

Por ejemplo, en un sistema de gestión de productos, se podrían almacenar códigos numéricos en un árbol de búsqueda. Si los productos se cargan en orden creciente, un ABB común puede quedar inclinado hacia la derecha. En un AVL, cada inserción revisa el balance y, si corresponde, aplica una rotación para evitar que la altura siga creciendo de manera lineal.

### Rotaciones como mecanismo de corrección

Una **rotación** es una reorganización local de nodos destinada a recuperar el equilibrio sin perder la propiedad de búsqueda. La rotación no cambia el conjunto de valores ni altera el orden lógico del ABB. Solo modifica las relaciones padre-hijo dentro de una pequeña zona del árbol.

La idea puede entenderse con tres valores ordenados: 10, 20 y 30. Si la estructura queda como 30 con hijo izquierdo 20, y 20 con hijo izquierdo 10, el árbol está inclinado hacia la izquierda. Una rotación derecha permite que 20 suba, 10 quede a su izquierda y 30 quede a su derecha. El orden sigue siendo correcto:  $10 < 20 < 30$ .

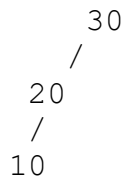
La documentación de AVL indica que, luego de una inserción o eliminación, debe verificarse que la diferencia de alturas no exceda 1, y que la corrección se realiza mediante rotaciones; con una o a lo sumo dos rotaciones se puede volver a balancear el árbol.

### Rotación simple derecha: caso izquierda-izquierda

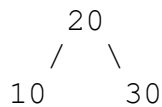
El caso izquierda-izquierda aparece cuando un nodo queda desbalanceado hacia la izquierda porque la inserción se produjo en el subárbol izquierdo de su hijo izquierdo. Con la convención adoptada, el nodo desbalanceado tendrá factor de equilibrio -2.

Ejemplo de inserción: 30, 20, 10.

La estructura queda así:



El nodo 30 queda desbalanceado hacia la izquierda. La corrección es una rotación simple derecha. Después de rotar:



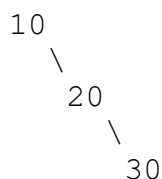
El nodo intermedio sube y la altura del subárbol disminuye. La propiedad de ABB se mantiene porque 10 sigue siendo menor que 20, y 30 sigue siendo mayor que 20.

### Rotación simple izquierda: caso derecha-derecha

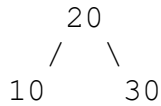
El caso derecha-derecha aparece cuando un nodo queda desbalanceado hacia la derecha porque la inserción se produjo en el subárbol derecho de su hijo derecho. Con la convención adoptada, el nodo desbalanceado tendrá factor de equilibrio +2.

Ejemplo de inserción: 10, 20, 30.

La estructura queda así:



El nodo 10 queda desbalanceado hacia la derecha. La corrección es una rotación simple izquierda. Después de rotar:

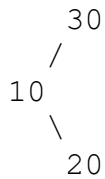


Nuevamente, el nodo intermedio sube y se recupera la altura balanceada del subárbol.

### Rotación doble izquierda-derecha

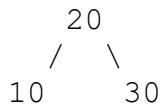
El caso izquierda-derecha aparece cuando el nodo se desbalancea hacia la izquierda, pero el nuevo valor se insertó en el subárbol derecho del hijo izquierdo. La forma ya no es una línea, sino un zigzag. Una sola rotación no alcanza porque primero hay que transformar el zigzag en una forma lineal.

Ejemplo de inserción: 30, 10, 20. La estructura inicial queda así:



El nodo 30 está cargado hacia la izquierda, pero el crecimiento dentro de su hijo izquierdo ocurrió hacia la derecha. La corrección se realiza en dos pasos. Primero, se aplica una rotación izquierda sobre 10. Luego, se aplica una rotación derecha sobre 30.

El resultado final es:



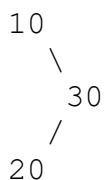
La rotación doble puede entenderse como una preparación seguida de una corrección. La primera rotación alinea la estructura; la segunda recupera el equilibrio.

### Rotación doble derecha-izquierda

El caso derecha-izquierda aparece cuando el nodo se desbalancea hacia la derecha, pero el nuevo valor se insertó en el subárbol izquierdo del hijo derecho.

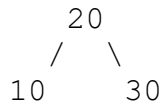
Ejemplo de inserción: 10, 30, 20.

La estructura inicial queda así:



El nodo 10 está cargado hacia la derecha, pero el crecimiento dentro de su hijo derecho ocurrió hacia la izquierda. La corrección también requiere dos pasos. Primero, se aplica una rotación derecha sobre 30. Luego, se aplica una rotación izquierda sobre 10.

El resultado final es:



Este caso es simétrico al izquierda-derecha. La forma visual del árbol permite decidir con más seguridad que la memorización de nombres.

### Criterio práctico para decidir la rotación

La rotación adecuada se decide observando dos aspectos: el lado hacia el que se desbalanceó el nodo y el lado por el que creció el hijo responsable del desbalance.

| Situación | Forma del crecimiento        | Rotación                |
|-----------|------------------------------|-------------------------|
| Factor -2 | Izquierda del hijo izquierdo | Simple derecha          |
| Factor +2 | Derecha del hijo derecho     | Simple izquierda        |
| Factor -2 | Derecha del hijo izquierdo   | Doble izquierda-derecha |
| Factor +2 | Izquierda del hijo derecho   | Doble derecha-izquierda |

Este criterio permite evitar un error frecuente: aplicar una rotación simple cuando el problema tiene forma de zigzag. Si el crecimiento cambia de dirección, la rotación debe ser doble.

### Implementación esencial de un AVL

La implementación de AVL requiere ampliar la estructura básica del nodo ABB. En un ABB simple, el nodo contiene el valor y las referencias a sus hijos. En la implementación de cátedra, el nodo ABB se define con `info`, `hijoIzq` e `hijoDer`, y la clase ABB implementa operaciones como `Raiz`, `ArbolVacio`, `AgregarElem` y `EliminarElem`.

En AVL, además del valor y las referencias a los hijos, se necesita registrar o calcular la altura del nodo. Guardar la altura como atributo simplifica el cálculo del factor de equilibrio.

```

class NodoAVL {
    int valor;
    int altura;
    NodoAVL izquierdo;
    NodoAVL derecho;

    NodoAVL(int valor) {
  
```

```

    this.valor = valor;
    this.altura = 1;
    this.izquierdo = null;
    this.derecho = null;
}
}

```

La altura se toma como 1 para un nodo recién creado y 0 para una referencia nula. Esta convención es habitual en implementaciones porque simplifica el cálculo.

```

int altura(NodoAVL nodo) {
    if (nodo == null) {
        return 0;
    }
    return nodo.altura;
}

```

El factor de equilibrio se calcula comparando la altura del subárbol derecho con la altura del subárbol izquierdo.

```

int factorEquilibrio(NodoAVL nodo) {
    if (nodo == null) {
        return 0;
    }
    return altura(nodo.derecho) - altura(nodo.izquierdo);
}

```

Después de insertar o rotar, la altura de los nodos afectados debe actualizarse.

```

void actualizarAltura(NodoAVL nodo) {
    nodo.altura = 1 + Math.max(altura(nodo.izquierdo), altura(nodo.derecho));
}

```

Este método es esencial. Si la altura no se actualiza correctamente, el árbol puede tomar decisiones de balance incorrectas. La estructura puede parecer bien conectada, pero el diagnóstico de equilibrio quedará desactualizado.

### Implementación de rotaciones simples

La rotación derecha corrige el caso izquierda-izquierda. Supone que existe un nodo y desbalanceado hacia la izquierda. Su hijo izquierdo  $x$  sube, y baja hacia la derecha, y el subárbol intermedio se reubica como hijo izquierdo de  $y$ .

```

NodoAVL rotarDerecha(NodoAVL y) {
    NodoAVL x = y.izquierdo;
    NodoAVL t2 = x.derecho;
}

```

```
x.derecho = y;
y.izquierdo = t2;

actualizarAltura(y);
actualizarAltura(x);

return x;
}
```

La rotación izquierda corrige el caso derecha-derecha. Supone que existe un nodo x desbalanceado hacia la derecha. Su hijo derecho y sube, x baja hacia la izquierda, y el subárbol intermedio se reubica como hijo derecho de x.

```
NodoAVL rotarIzquierda(NodoAVL x) {
    NodoAVL y = x.derecho;
    NodoAVL t2 = y.izquierdo;

    y.izquierdo = x;
    x.derecho = t2;

    actualizarAltura(x);
    actualizarAltura(y);

    return y;
}
```

Las rotaciones dobles no necesitan implementarse como operaciones completamente nuevas. Se componen a partir de las rotaciones simples. En izquierda-derecha se rota primero a la izquierda el hijo izquierdo y luego a la derecha el nodo desbalanceado. En derecha-izquierda se rota primero a la derecha el hijo derecho y luego a la izquierda el nodo desbalanceado.

### Inserción balanceada en AVL

La inserción AVL integra tres lógicas: insertar como ABB, actualizar altura y corregir balance. El siguiente código muestra una implementación mínima orientada al concepto.

```
NodoAVL insertar(NodoAVL nodo, int valor) {
    if (nodo == null) {
        return new NodoAVL(valor);
    }

    if (valor < nodo.valor) {
        nodo.izquierdo = insertar(nodo.izquierdo, valor);
    } else if (valor > nodo.valor) {
        nodo.derecho = insertar(nodo.derecho, valor);
    }
}
```

```

    } else {
        return nodo; // no se insertan repetidos
    }

    actualizarAltura(nodo);

    int fe = factorEquilibrio(nodo);

    // Caso izquierda-izquierda
    if (fe < -1 && valor < nodo.izquierdo.valor) {
        return rotarDerecha(nodo);
    }

    // Caso derecha-derecha
    if (fe > 1 && valor > nodo.derecho.valor) {
        return rotarIzquierda(nodo);
    }

    // Caso izquierda-derecha
    if (fe < -1 && valor > nodo.izquierdo.valor) {
        nodo.izquierdo = rotarIzquierda(nodo.izquierdo);
        return rotarDerecha(nodo);
    }

    // Caso derecha-izquierda
    if (fe > 1 && valor < nodo.derecho.valor) {
        nodo.derecho = rotarDerecha(nodo.derecho);
        return rotarIzquierda(nodo);
    }

    return nodo;
}

```

Este código permite observar la estructura completa del razonamiento. Primero se baja recursivamente por el árbol como en ABB. Luego, al volver de la recursión, se actualiza la altura. Después se calcula el factor de equilibrio y se decide si corresponde rotar. Finalmente se retorna la nueva raíz del subárbol, porque una rotación puede cambiar qué nodo queda arriba.

El retorno de la raíz del subárbol es un detalle importante. En una rotación, el nodo que antes era raíz local puede dejar de serlo. Si no se retorna y reasigna correctamente, el árbol puede perder conexiones.

### Eliminación en ABB y eliminación en AVL

La eliminación en ABB consiste en localizar el nodo a eliminar y resolver uno de varios casos. Si el nodo es hoja, se elimina directamente. Si tiene subárbol izquierdo, puede

reemplazarse por el mayor valor del subárbol izquierdo, llamado predecesor. Si no tiene subárbol izquierdo pero tiene subárbol derecho, puede reemplazarse por el menor valor del subárbol derecho, llamado sucesor. El apunte de cátedra implementa esta lógica en `EliminarElem(int x)`, utilizando métodos auxiliares como `mayor` y `menor`.

En AVL, la eliminación parte de la misma lógica del ABB, pero no termina allí. Luego de eliminar, la estructura puede quedar desbalanceada. Por eso, debe actualizarse la altura de los nodos afectados y aplicarse el mismo criterio de balance que en la inserción.

La lógica general puede representarse así:

```
eliminarComoABB(valor);
actualizarAltura(nodo);
calcularFactorEquilibrio(nodo);
aplicarRotacionSiCorresponde();
```

La eliminación AVL suele ser más compleja que la inserción porque el desbalance puede propagarse hacia arriba en más de un nivel. Por eso, desde el punto de vista conceptual, lo fundamental es comprender que eliminar en AVL significa dos cosas: primero eliminar respetando la propiedad de ABB, y luego restaurar el equilibrio AVL. La documentación de AVL indica que tanto inserción como eliminación siguen la lógica de ABB, pero deben conservar la condición de equilibrio mediante verificaciones y rotaciones.

### Costo computacional

El objetivo principal de AVL es controlar la altura. En un ABB común, el costo de búsqueda, inserción y eliminación depende de  $h$ . Si el árbol queda balanceado,  $h$  se aproxima a  $\log n$ . Si queda degenerado,  $h$  puede aproximarse a  $n$ .

AVL evita esa degradación mediante la regla de equilibrio. Al garantizar que ningún nodo tenga subárboles con diferencia de altura mayor que 1, impide que la estructura crezca como una cadena. Por eso, las operaciones principales se mantienen en  **$O(\log n)$** . El material de AVL afirma que la búsqueda queda acotada en  $O(\log n)$ , y que inserción y eliminación también conservan ese orden por la forma en que el árbol se mantiene balanceado.

Este costo tiene un precio: cada inserción o eliminación requiere trabajo adicional para actualizar alturas, calcular factores de equilibrio y eventualmente rotar. Sin embargo, ese costo adicional permite evitar el peor caso del ABB común. En sistemas donde se realizan búsquedas frecuentes, esta garantía suele justificar el mantenimiento del balance.

### Cuándo conviene usar AVL

AVL resulta adecuado cuando se necesita mantener búsquedas eficientes y predecibles sobre datos que cambian dinámicamente. Puede utilizarse en sistemas donde se insertan y eliminan valores, pero se requiere que las búsquedas sigan siendo rápidas.

Ejemplos verosímiles incluyen índices en memoria para códigos de productos, identificadores de usuarios, números de cuenta, legajos de estudiantes o claves internas de operaciones. En todos esos casos, un ABB común podría degradarse si los datos ingresan en un orden desfavorable. AVL reduce ese riesgo porque reequilibra la estructura.

No siempre AVL es la única opción. Si los datos están en disco o en almacenamiento externo de gran volumen, otras estructuras como árboles B pueden ser más apropiadas, porque agrupan muchas claves por nodo y reducen accesos a bloques. El anexo sobre árboles B señala que, aunque AVL ayuda a resolver el balance en memoria, no resulta eficiente cuando el volumen obliga a trabajar con disco, ya que cada acceso puede leer un bloque del que se usa solo una parte pequeña.

La diferencia es de criterio de diseño. AVL es útil para búsqueda balanceada en memoria. Los árboles B están pensados para grandes volúmenes y almacenamiento externo. Ambos buscan controlar la altura, pero responden a problemas de implementación distintos.

### Errores frecuentes

Un error habitual es pensar que AVL ordena los datos de una manera diferente al ABB. No es así. La propiedad de orden es la misma. AVL agrega balance, no una nueva regla de ordenamiento.

Otro error frecuente es creer que toda inserción produce una rotación. Muchas inserciones no rompen la condición de equilibrio. La rotación solo se aplica cuando algún nodo queda con factor  $+2$  o  $-2$ .

También suele confundirse rotación simple con rotación doble. La rotación simple corresponde a un crecimiento lineal: izquierda-izquierda o derecha-derecha. La rotación doble corresponde a un crecimiento en zigzag: izquierda-derecha o derecha-izquierda.

Otro error importante es olvidarse de actualizar alturas después de modificar referencias. En una implementación AVL, rotar sin actualizar alturas deja la estructura visualmente conectada, pero lógicamente inconsistente para futuras operaciones.

Finalmente, no debe confundirse balance AVL con árbol perfectamente completo. Un AVL no exige que todos los niveles estén llenos. Exige que cada nodo mantenga una diferencia de altura máxima de una unidad entre sus subárboles.

### Síntesis conceptual

El Árbol AVL resuelve una limitación central del ABB: la posibilidad de que una estructura lógicamente correcta se vuelva operativamente ineficiente por desbalance. La propiedad de búsqueda permite decidir hacia dónde avanzar; la condición de equilibrio permite controlar la altura y sostener costos eficientes.

El factor de equilibrio es el instrumento de diagnóstico. Las rotaciones son el mecanismo de corrección. Las rotaciones simples resuelven crecimientos lineales; las rotaciones dobles resuelven crecimientos en zigzag. En todos los casos, se preserva la propiedad del ABB.

Desde el punto de vista de implementación, AVL exige almacenar o calcular alturas, actualizar esos valores después de cada modificación, calcular factores de equilibrio y retornar correctamente las nuevas raíces locales después de rotar. La inserción AVL es una extensión de la inserción ABB: primero se ubica el valor respetando el orden, luego se restaura el balance. La eliminación AVL también parte de la eliminación ABB, pero debe agregar actualización de alturas y rebalanceo posterior.

La relación entre estructura y costo es el eje del tema. Un ABB puede ser correcto y lento si su altura crece demasiado. Un AVL agrega reglas y rotaciones para evitar esa degradación, manteniendo búsqueda, inserción y eliminación en  **$O(\log n)$** .